



Claude Code & Claude Cowork

An honest tour

Agustín · UW-Madison

What are these tools?



Claude Code

Command-line agent that reads, writes, edits, and runs code directly in your repository.

Lives in the terminal. Aimed at developers.

Claude Cowork

Desktop agent that works on your files (Word, Excel, PDF, CSV...) and connects to Gmail, Slack, Drive, Calendar.

No terminal needed. Aimed at everyone.

Both are built on the Claude model family. Both are agentic: plan → act → observe → re-plan.

DEMO — Claude Code



Coffee Shops

DEMO — Claude Cowork



Project Notes

Pros — research workflows



- **Time to first result** drops dramatically. CSV → plot in minutes, not hours.
- **Simple and fast** for small tasks: renaming files, regenerating figures, parsing logs.
- **Automatic work done fast** If you have rutinary work that is repetitive and easy, you could automate it
- **Reading partner/search engine** drop a paper or codebase; ask focused questions faster than skimming.
- **Multi-language** R · Python · Bash · SQL · LaTeX. Example: write the program 'slow.R' in Fortran.
- **Reproducibility** writes README, Rmd, update GitHub, all this is faster/easier with this tool.

Pros



Claude Code

- Edits files in place — no copy-paste.
- Can run tests, good for troubleshooting.
- CLAUDE.md = project memory: claude improves over time by updating this file.
- Extensible: GitHub, GitLab, etc.
- **All that Claude Cowork does can be done by Claude Code**

Claude Cowork

- Lower barrier: colleagues who avoid the terminal can use it.



The things the tool will not tell you.

These were learned over months of real use, with many pitfalls and mistakes along the way.
My honest experience, without makeup, from a grad student to grad students



Logic bugs that look right

The code runs. It passes a first read. It is still wrong.

- Wrong grouping in dplyr / pandas → incorrect per-sample means.
- Animal IDs read as numeric — leading zeros silently lost.
- Default statistical parameters that don't match your experimental design.

In my experience, the most painful bugs in programing are the logic ones. The tool can introduce these bugs in a way where the final output doesn't necessarily seem odd. This is a truly dangerous behavior. This implies that you REALLY NEED to carefully read and understand the output code. Hence, you don't spend time writing code, instead, reading/checking code.



You will need to read complex codes

If you cannot read code, the tool will surpass you — and you lose control of your own project.

- Mistakes are encrypted: hidden inside code that looks fine.
- The agent does not free you from understanding code.
- As it is nowadays, the tool raises the floor, and raises it above some people's ceiling...



Convergence to the expected

The tool pushes the rationale to an average, expected solution.

- It may reduce creativity and authenticity.
- It may reduce novel approaches — the predecessors of new methodologies.
- May converge to an average of what was done before, not what needs to be done next.

"In science, the part of your work that matters the most is the part that is not in the training set. A tool that regresses you to the mean dictated by a training set could be against creativity and novel approaches"



Bias toward the popular answer

Answers are weighted toward the most frequent data in training set.

- Bias: Ask Claude for holiday destinations: you will likely get the most frequent and popular answers. Even if you try to narrow it down, you will still get frequent and popular destinations. Try it!
- Expectation: If you are the expert in your field, you are better than men. An average reflects central tendency, mean performance, rather than to extreme outcomes. Therefore, what we could have from an expected answer fitted from a massive training set?

"Unless you control what you want in a thin, specific manner, the code/concept you get will be functionally correct and utterly average. Somehow will be in the training set"



Less effort = shallower understanding

Less cognitive effort to get a result means a shallower understanding of the process.

- The value of doing your own GBLUP analysis the first time isn't the vector of solution for genomic breeding values.
- It's the knowledge you build. At the end of the day, this knowledge is encrypted in your own way of reasoning.
- Speed without depth produces fluent-looking work that collapses under questioning.

"Science is what we understand well enough to explain to a computer. Art is everything else we do."

— Donald E. Knuth



Be careful with homework

It will save you a lot of time and probably you will not truly learn the concept.

- Less cognitive effort → fewer connections → shallower understanding.
- The grade is not the point (despite I understand the pressure we have for good grades).
- The real point is that we need to learn, and this take time and friction



Dependency risk

You can build a real dependency on a tool whose price and availability are not under your control.

- Today's pricing and access are a business decision, not a law of physics.
- If access tripled in price tomorrow, or disappeared for a week — what stops?



There is an economic incentive layer

The companies pushing autonomous agents the hardest also own the compute.

- More automated agents = orders of magnitude more compute demand.
- The people selling the agents are the people selling the compute.
- The enthusiasm has a business model behind it. That's fine!! but I think is an important component to understand the snowball of AI we are living nowadays.



My verdict

More good than bad → rounding to the closest integer → it is a good tool!!

And it would be even better if you can mitigate its drawback given you are aware about them

Probably, soon, some of this limitations are going to disappear



"Use it as a tool, not as an oracle"

Keep your authenticity

Many novel discoveries where initially a beautiful and unexpected mistake!

Discuss, engage, think and challenge assumptions